Solo-Git Phase 2: Spec ↔ Code Coverage Matrix

Generated: November 3, 2025
Branch: phase-2-audit-refactor
Purpose: Documentation-driven audit and safe refactor

Table of Contents

Matrix Legend

Status Indicators

- ✅ **Implemented & Tested** - Feature complete with tests
- ⚠️ **Implemented, Low Test Coverage** - Feature exists but needs more tests
- 🔴 **Missing** - Feature documented but not implemented
- 📝 **Documented Only** - Spec exists, no code
- 🔄 **Partial** - Partially implemented

Test Coverage Levels

- **High**: ≥90% coverage
- **Medium**: 70-89% coverage
- **Low**: 50-69% coverage
- **Critical**: <50% coverage

Core Repository Management

REQ-REPO-001: Repository Initialization

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
Initialize from ZIP	README.md, FEA-TURES.md	`engines/git_engine.py::initial-ize_from_zip()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Initialize from Git URL	README.md, FEA-TURES.md	`engines/git_engine.py::initial-ize_from_git()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Initialize from scratch	FEA-TURES.md	`engines/git_engine.py::initial-ize_repository()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Automatic `.sologit/` creation	ARCHITEC-TURE.md	`engines/git_engine.py::_create_sologit_struc-ture()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Git reposit-ory initial-ization	FEA-TURES.md	`engines/git_engine.py::_ensure_git_repo()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Repository metadata tracking	ARCHITEC-TURE.md	`core/re-posit-ory.py::Repository`	`tests/test_repository.py`	✅	High (100%)	None

Module: `sologit/engines/git_engine.py` (1788 lines)
Classes: GitEngine, GitEngineError, RepositoryNotFoundError

Test Files: test_git_engine.py (56+ tests)
Overall Status*: ✅ Complete

REQ-REPO-002: Repository Operations

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
List repositories	README.md	`engines/git_engine.py::list_repositories()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Get repository info	README.md	`engines/git_engine.py::get_repository_info()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Repository state sync	ARCHITECTURE.md	`state/git_sync.py::GitStateSync`	`tests/test_git_sync.py`	✅	Medium (75%)	More edge case tests needed
File change detection	ARCHITECTURE.md	`engines/git_engine.py::get_file_changes()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Diff generation	FEATURES.md	`engines/git_engine.py::get_diff()`	`tests/test_git_engine*.py`	✅	High (90%)	None

Module: `sologit/engines/git_engine.py`, `sologit/state/git_sync.py`
Test Coverage: 85% average
Overall Status: ✅ Mostly Complete, needs edge case tests

Workpad System

REQ-WORKPAD-001: Workpad Lifecycle

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
Create workpad	README.md, FEA-TURES.md	`engines/git_engine.py::create_workpad()`	`tests/test_git_engine*.py`	✅	High (90%)	None
List work-pads	README.md, FEA-TURES.md	`engines/git_engine.py::list_work-pads()`	`tests/test_git_engine*.py`	✅	High (100%)	None
Get work-pad info	README.md	`engines/git_engine.py::get_work-pad_info()`	`tests/test_git_engine*.py`	✅	High (100%)	None
Delete workpad	README.md, FEA-TURES.md	`engines/git_engine.py::delete_workpad()`	`tests/test_git_engine*.py`	✅	High (100%)	None
Workpad status tracking	ARCHITEC-TURE.md	`core/work-pad.py::Workpad`	`tests/test_workpad.py`	✅	High (100%)	None
Ephemeral namespace (`pads/`)	ARCHITEC-TURE.md	`engines/git_engine.py::_create_workpad_branch()`	`tests/test_git_engine*.py`	✅	High (90%)	None

Module: `sologit/core/workpad.py` (159 lines)
Classes: Workpad, Checkpoint, Snapshot
Test Files: test_workpad.py, test_git_engine.py
Overall Status*: ✅ Complete

REQ-WORKPAD-002: Checkpointing

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Create checkpoint	FEATURES.md	`engines/git_engine.py::checkpoint_workpad()`	`tests/test_git_engine*.py`	✅	High (90%)	None
View checkpoint history	FEATURES.md	`engines/git_engine.py::get_workpad_history()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Revert to checkpoint	FEATURES.md	`engines/git_engine.py::revert_to_checkpoint()`	`tests/test_git_engine*.py`	⚠️	Medium (70%)	Needs more complex revert scenarios
File snapshot on checkpoint	ARCHITECTURE.md	`core/workpad.py::Snapshot`	`tests/test_workpad.py`	✅	High (100%)	None
Automatic timestamp tracking	FEATURES.md	`core/workpad.py::Checkpoint`	`tests/test_workpad.py`	✅	High (100%)	None

Overall Status: ✅ Mostly Complete, needs revert tests

REQ-WORKPAD-003: Promotion (Fast-Forward Only)

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Promote workpad to trunk	README.md, FEATURES.md	`engines/git_engine.py::promote_workpad()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Fast-forward only enforcement	ARCHITECTURE.md	`engines/git_engine.py::_fast_forward_merge()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Conflict detection	FEATURES.md	`engines/git_engine.py::detect_conflicts()`	`tests/test_git_engine*.py`	✅	High (90%)	None
Automatic rebase on trunk move	FEATURES.md	`engines/git_engine.py::rebase_workpad()`	`tests/test_git_engine*.py`	⚠️	Medium (70%)	Needs more rebase conflict tests
Promotion rules validation	FEATURES.md	`workflows/promotion_gate.py::PromotionGate`	`tests/test_promotion_gate.py`	✅	High (80%)	None
Auto-promotion on green tests	FEATURES.md	`workflows/auto_merge.py::AutoMergeWorkflow`	`tests/test_auto_merge.py`	✅	High (80%)	None

Module: `sologit/workflows/promotion_gate.py` (248 lines)
Classes: PromotionGate, PromotionDecision, PromotionRules
Test Files: test_promotion_gate.py (13 tests)
Overall Status: ✅ Mostly Complete, needs rebase edge cases

AI Orchestration

REQ-AI-001: Multi-Model Routing

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Three-tier model system	README.md, ARCHITECTURE.md	`orchestration/model_router.py::ModelRouter`	`tests/test_model_router.py`	✅	High (89%)	None
Planning tier (GPT-4, Claude)	README.md	`orchestration/model_router.py::ModelTier.PLANNING`	`tests/test_model_router.py`	✅	High (89%)	None
Coding tier (DeepSeek, CodeLlama)	README.md	`orchestration/model_router.py::ModelTier.CODING`	`tests/test_model_router.py`	✅	High (89%)	None
Fast tier (Llama 8B, Gemma 9B)	README.md	`orchestration/model_router.py::ModelTier.FAST`	`tests/test_model_router.py`	✅	High (89%)	None
Complexity-based routing	ARCHITECTURE.md	`orchestration/model_router.py::_calculate_complexity()`	`tests/test_model_router.py`	✅	High (89%)	None
Security keyword escalation	README.md	`orchestration/model_router.py::_has_security_keywords()`	`tests/test_model_router.py`	✅	High (89%)	None

Module: `sologit/orchestration/model_router.py` (514 lines)
Classes: ModelRouter, ModelTier, ComplexityMetrics
Test Files: test_model_router.py (13 tests)
Overall Status: ✅ Complete

REQ-AI-002: Provider Architecture (Abacus-First)

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Abacus.AI primary provider	ACTION_PLAN.pdf	`orchestration/providers/abacus_adapter.py`	`tests/test_providers.py`	✅	High (85%)	None
OpenAI fallback provider	ACTION_PLAN.pdf	`orchestration/providers/openai_adapter.py`	`tests/test_providers.py`	✅	High (85%)	None
Anthropic fallback provider	ACTION_PLAN.pdf	`orchestration/providers/anthropic_adapter.py`	`tests/test_providers.py`	✅	High (85%)	None
Automatic failover	ACTION_PLAN.pdf	`orchestration/routing_policy.py::PolicyEngine`	`tests/test_routing_policy.py`	✅	High (85%)	None
Health checking with caching	ACTION_PLAN.pdf	`orchestration/providers/*_adapter.py::is_available()`	`tests/test_providers.py`	⚠️	Medium (70%)	Cache expiry tests needed
Unified provider interface	ACTION_PLAN.pdf	`orchestration/providers/__init__.py::ProviderAdapter`	`tests/test_providers.py`	✅	High (85%)	None

Module: `sologit/orchestration/providers/` (3 adapter files)
Test Files: test_providers.py, test_routing_policy.py
Overall Status: ✅ Mostly Complete, needs cache tests

REQ-AI-003: Commit Message Generation

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
AI-assisted commit messages	ACTION_PLAN.pdf	`orchestration/commit_message_generator.py`	`tests/test_commit_message_generator.py`	✅	High (85%)	None
Conventional Commits format	ACTION_PLAN.pdf	`orchestration/commit_message_generator.py::_build_system_prompt()`	`tests/test_commit_message_generator.py`	✅	High (85%)	None
Free-form option	ACTION_PLAN.pdf	`orchestration/commit_message_generator.py::CommitMessageRequest.conventional_commit`	`tests/test_commit_message_generator.py`	✅	High (85%)	None
Interactive editing	ACTION_PLAN.pdf	`cli/integrated_commands.py::generate_commit_message()`	`tests/test_cli_integrated.py`	⚠️	Low (40%)	CLI integration tests needed
Metadata tracking	ACTION_PLAN.pdf	`orchestration/commit_message_generator.py::CommitMessageResponse`	`tests/test_commit_message_generator.py`	✅	High (85%)	None
Telemetry		`orchestration/tele-`		⚠️	Medium (65%)	More telemetry

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
	AC-TION_PLAN .pdf	`metry.py::Tele-metryCol-lector`	`tests/test_telemetry.py`			tests needed

Module: `sologit/orchestration/commit_message_generator.py` (201 lines)
Test Files: test_commit_message_generator.py
Overall Status: ✅ Mostly Complete, needs CLI tests

REQ-AI-004: Planning & Code Generation

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
AI code planning	README.md, ARCHITECTURE.md	`orchestration/planning_engine.py::PlanningEngine`	`tests/test_planning_engine.py`	✅	High (79%)	None
Structured plan output	ARCHITECTURE.md	`orchestration/planning_engine.py::CodePlan`	`tests/test_planning_engine.py`	✅	High (79%)	None
Repository context integration	ARCHITECTURE.md	`orchestration/planning_engine.py::plan_task()`	`tests/test_planning_engine.py`	✅	High (79%)	None
Patch generation	README.md	`orchestration/code_generator.py::CodeGenerator`	`tests/test_code_generator.py`	✅	High (84%)	None
Multiple specialist coders	ARCHITECTURE.md	`orchestration/code_generator.py::_select_coder()`	`tests/test_code_generator.py`	✅	High (84%)	None
Unified diff format	ARCHITECTURE.md	`orchestration/code_generator.py::generate_patch()`	`tests/test_code_generator.py`	✅	High (84%)	None

Module: `sologit/orchestration/planning_engine.py` (391 lines), `code_generator.py` (409 lines)
Test Files: test_planning_engine.py (12 tests), test_code_generator.py (14 tests)
Overall Status: ✅ Complete

REQ-AI-005: Cost Management

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Budget tracking by model	FEATURES.md	`orchestration/cost_guard.py::CostTracker`	`tests/test_cost_guard.py`	✅	High (93%)	None
Daily spending caps	FEATURES.md	`orchestration/cost_guard.py::CostGuard.check_budget()`	`tests/test_cost_guard.py`	✅	High (93%)	None
Alert thresholds	FEATURES.md	`orchestration/cost_guard.py::CostGuard.should_alert()`	`tests/test_cost_guard.py`	✅	High (93%)	None
Per-workpad cost tracking	FEATURES.md	`orchestration/cost_guard.py::track_workpad_cost()`	`tests/test_cost_guard.py`	⚠️	Medium (70%)	Workpad-specific tracking needs tests
Token usage monitoring	ARCHITECTURE.md	`orchestration/cost_guard.py::TokenUsage`	`tests/test_cost_guard.py`	✅	High (93%)	None
Spending reports	FEATURES.md	`orchestration/cost_guard.py::get_spending_report()`	`tests/test_cost_guard.py`	✅	High (93%)	None

Module: `sologit/orchestration/cost_guard.py` (441 lines)
Test Files: test_cost_guard.py (14 tests)
Overall Status: ✅ Mostly Complete, needs workpad tracking tests

---

Test Orchestration

REQ-TEST-001: Test Execution

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
Fast tests (<30s)	README.md, FEATURES.md	`engines/test_orchestrator.py::run_tests(target='fast')`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
Full test suite	README.md, FEATURES.md	`engines/test_orchestrator.py::run_tests(target='full')`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
Sandboxed execution	FEATURES.md	`engines/test_orchestrator.py::_run_in_sandbox()`	`tests/test_test_orchestrator.py`	✅	High (100%)	Now using subprocess sandboxing
Parallel test execution	FEATURES.md	`engines/test_orchestrator.py::run_parallel()`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
Timeout enforcement	FEATURES.md	`engines/test_orchestrator.py::_enforce_timeout()`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
Real-time progress	FEATURES.md	`engines/test_orchestrator.py::_emit_progress()`	`tests/test_test_orchestrator.py`	✅	High (100%)	None

Module: `sologit/engines/test_orchestrator.py` (874 lines)
Classes: TestOrchestrator, TestResult, TestConfig
Test Files: test_test_orchestrator.py (20 tests)
Overall Status: ✅ Complete

REQ-TEST-002: Test Frameworks Support

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
pytest (Python)	FEATURES.md	`engines/test_orchestrator.py::_detect_framework()`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
unittest (Python)	FEATURES.md	`engines/test_orchestrator.py::_detect_framework()`	`tests/test_test_orchestrator.py`	✅	High (100%)	None
jest (JavaScript)	FEATURES.md	`engines/test_orchestrator.py::_detect_framework()`	`tests/test_test_orchestrator.py`	⚠️	Low (50%)	Needs actual jest tests
mocha (JavaScript)	FEATURES.md	`engines/test_orchestrator.py::_detect_framework()`	`tests/test_test_orchestrator.py`	⚠️	Low (50%)	Needs actual mocha tests
Go test (Go)	FEATURES.md	`engines/test_orchestrator.py::_detect_framework()`	`tests/test_test_orchestrator.py`	⚠️	Low (50%)	Needs actual go tests
Custom framework support	FEATURES.md	`config/manager.py::TestConfig`	`tests/test_config.py`	✅	High (85%)	None

Overall Status: ⚠️ Core complete, needs more framework-specific tests

---

REQ-TEST-003: Test Analysis

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
9-category failure classification	FEATURES.md	`analysis/test_analyzer.py::FailureCategory`	`tests/test_test_analyzer.py`	✅	High (90%)	None
ASSERTION_FAILURE	FEATURES.md	`analysis/test_analyzer.py::classify_failure()`	`tests/test_test_analyzer.py`	✅	High (90%)	None
DEPENDENCY_ERROR	FEATURES.md	`analysis/test_analyzer.py::classify_failure()`	`tests/test_test_analyzer.py`	✅	High (90%)	None
TIMEOUT	FEATURES.md	`analysis/test_analyzer.py::classify_failure()`	`tests/test_test_analyzer.py`	✅	High (90%)	None
SETUP_ERROR	FEATURES.md	`analysis/test_analyzer.py::classify_failure()`	`tests/test_test_analyzer.py`	✅	High (90%)	None
Test result aggregation	FEATURES.md	`analysis/test_analyzer.py::aggregate_results()`	`tests/test_test_analyzer.py`	✅	High (90%)	None
Failure pattern detection	ARCHITECTURE.md	`analysis/test_analyzer.py::id`	`tests/test_test_`	✅	High (90%)	None

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
		`enti-fy_pattern s()`	`analyz-er.py`			
AI-powered diagnosis	ARCHITEC-TURE.md	`analysis/test_analyzer.py::analyze_with_ai()`	`tests/test_test_analyz-er.py`	⚠️	Medium (70%)	Needs more AI in-tegration tests

Module: `sologit/analysis/test_analyzer.py` (426 lines)
Classes: TestAnalyzer, FailureCategory, TestAnalysis
Test Files: test_test_analyzer.py (19 tests)
Overall Status: ✅ Mostly Complete, needs AI tests

Workflow Automation

REQ-WORKFLOW-001: Auto-Merge Pipeline

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Complete auto-merge workflow	FEATURES.md	`workflows/auto_merge.py::AutoMergeWorkflow`	`tests/test_auto_merge.py`	✅	High (80%)	None
Test execution step	FEATURES.md	`workflows/auto_merge.py::_run_tests()`	`tests/test_auto_merge.py`	✅	High (80%)	None
Failure analysis step	FEATURES.md	`workflows/auto_merge.py::_analyze_failures()`	`tests/test_auto_merge.py`	✅	High (80%)	None
Promotion gate step	FEATURES.md	`workflows/auto_merge.py::_check_promotion_gate()`	`tests/test_auto_merge.py`	✅	High (80%)	None
Auto-promotion step	FEATURES.md	`workflows/auto_merge.py::_promote_workpad()`	`tests/test_auto_merge.py`	✅	High (80%)	None
CI smoke tests step	FEATURES.md	`workflows/auto_merge.py::_run_ci_smoke_tests()`	`tests/test_auto_merge.py`	⚠️	Medium (65%)	More CI integration tests needed
Auto-rollback step	FEATURES.md	`workflows/auto_merge.py::_rollback_on_failure()`	`tests/test_auto_merge.py`	⚠️	Medium (65%)	More rollback tests needed

Module: `sologit/workflows/auto_merge.py` (592 lines)
Classes: AutoMergeWorkflow, AutoMergeResult
Test Files: test_auto_merge.py
Overall Status: ✅ Core complete, needs more edge case tests

REQ-WORKFLOW-002: Promotion Rules

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
Configurable promotion rules	FEATURES.md	`workflows/promotion_gate.py::PromotionRules`	`tests/test_promotion_gate.py`	✅	High (80%)	None
`require_passing_tests`	FEATURES.md	`workflows/promotion_gate.py::_check_tests_passed()`	`tests/test_promotion_gate.py`	✅	High (80%)	None
`fast_forward_only`	FEATURES.md	`workflows/promotion_gate.py::_check_fast_forward()`	`tests/test_promotion_gate.py`	✅	High (80%)	None
`max_changed_files`	FEATURES.md	`workflows/promotion_gate.py::_check_file_count()`	`tests/test_promotion_gate.py`	✅	High (80%)	None
`min_test_coverage`	FEATURES.md	`workflows/promotion_gate.py::_check_coverage()`	`tests/test_promotion_gate.py`	⚠️	Medium (70%)	Coverage calculation needs tests
`blocked_files`	FEATURES.md	`workflows/promotion_gate.py::_check_blocked_files()`	`tests/test_promotion_gate.py`	✅	High (80%)	None
	FEATURES.md	`workflows/promo-`	`tests/test_promo`	⚠️	Medium (65%)	

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
Manual override support		`tion_gate.py::allow_override`	`tion_gate.py`			Override tests needed

Module: `sologit/workflows/promotion_gate.py` (248 lines)
Test Files: test_promotion_gate.py (13 tests)
Overall Status: ✅ Mostly Complete, needs coverage & override tests

---

REQ-WORKFLOW-003: CI Integration

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
Post-promotion smoke tests	FEATURES.md	`workflows/ci_orchestrator.py::run_smoke_tests()`	`tests/test_ci_orchestrator.py`	✅	High (85%)	None
Jenkins integration	ARCHITECTURE.md	`workflows/ci_orchestrator.py::trigger_jenkins_build()`	`tests/test_ci_orchestrator.py`	⚠️	Low (50%)	Needs actual Jenkins tests
GitHub Actions integration	FEATURES.md	`workflows/ci_orchestrator.py::trigger_github_action()`	`tests/test_ci_orchestrator.py`	🔴	None	Not implemented
Deployment simulation	FEATURES.md	`workflows/ci_orchestrator.py::simulate_deployment()`	None	🔴	None	Not implemented
Security scanning hooks	FEATURES.md	`workflows/ci_orchestrator.py::run_security_scan()`	None	🔴	None	Not implemented

Module: `sologit/workflows/ci_orchestrator.py` (273 lines)
Classes: CIOrchestrator, CIResult

Test Files: test_ci_orchestrator.py
Overall Status: ⚠️ Core complete, missing GitHub Actions & security features

---

REQ-WORKFLOW-004: Rollback Handler

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Automatic rollback on CI failure	FEATURES.md	`workflows/rollback_handler.py::RollbackHandler`	`tests/test_rollback_handler.py`	✅	Medium (62%)	None
Workpad recreation from backup	FEATURES.md	`workflows/rollback_handler.py::recreate_workpad()`	`tests/test_rollback_handler.py`	⚠️	Low (50%)	More recreation tests needed
State restoration	FEATURES.md	`workflows/rollback_handler.py::restore_state()`	`tests/test_rollback_handler.py`	⚠️	Low (50%)	More state tests needed
Git tree reset	FEATURES.md	`workflows/rollback_handler.py::reset_git_tree()`	`tests/test_rollback_handler.py`	✅	Medium (62%)	None
Notification system	FEATURES.md	`workflows/rollback_handler.py::send_notification()`	None	🔴	None	Not implemented

Module: `sologit/workflows/rollback_handler.py` (225 lines)
Classes: RollbackHandler, RollbackResult

Test Files: test_rollback_handler.py
Overall Status: ⚠️ Core complete, needs more tests & notifications

---

User Interfaces

REQ-UI-001: CLI (Command-Line Interface)

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Rich formatting	README.md, FEATURES.md	`ui/formatter.py::RichFormatter`	`tests/test_formatter.py`	✅	High (85%)	None
Interactive prompts	FEATURES.md	`cli/commands.py::*`	`tests/test_cli*.py`	⚠️	Low (30%)	CLI tests needed
Command aliases	FEATURES.md	`cli/main.py`	None	🔴	None	Not implemented
Shell completion	FEATURES.md	`ui/autocomplete.py::SoloGitCompleter`	`tests/test_autocomplete.py`	⚠️	Medium (60%)	More completion tests needed
JSON/CSV output	FEATURES.md	`cli/commands.py::--json, --csv`	None	⚠️	Low (20%)	Partial implementation
Progress bars and spinners	FEATURES.md	`ui/formatter.py::ProgressContext`	`tests/test_formatter.py`	✅	High (85%)	None

Modules: `sologit/cli/` (6 files, ~5000 lines total)
Test Files: test_cli.py, test_formatter.py
Overall Status*: ⚠️ Core complete, needs more CLI integration tests

---

REQ-UI-002: TUI (Terminal UI)

Require-ment	Spec Source	Imple-menta-tion	Tests	Status	Coverage	Gaps/ Notes
Textual framework integration	README.md	`ui/heaven_tui.py::HeavenTUI`	`tests/test_heaven_tui.py`	✅	Medium (60%)	None
Workpad list panel	FEATURES.md	`ui/heaven_tui.py::WorkpadPanel`	`tests/test_heaven_tui.py`	⚠️	Low (40%)	More UI tests needed
File tree browser	FEATURES.md	`ui/file_tree.py::FileTreeWidget`	`tests/test_file_tree.py`	⚠️	Low (50%)	More tree tests needed
Commit history graph (ASCII)	FEATURES.md	`ui/graph.py::CommitGraphRenderer`	`tests/test_graph.py`	⚠️	Low (45%)	More graph tests needed
Test result viewer	FEATURES.md	`ui/test_runner.py::TestRunnerWidget`	`tests/test_test_runner.py`	⚠️	Medium (55%)	More test UI tests needed
AI chat panel	FEATURES.md	`ui/heaven_tui.py::AIActivityPanel`	None	⚠️	Low (30%)	Minimal testing
Command palette (fuzzy search)	FEATURES.md	`ui/command_palette.py::CommandPaletteScreen`	`tests/test_command_palette.py`	⚠️	Medium (60%)	More palette tests needed
Keyboard shortcuts	FEATURES.md	`ui/shortcuts.py::Shortcut`	`tests/test_shortcuts.py`	⚠️	Low (50%)	More shortcut tests needed

Modules: `sologit/ui/` (14 files, ~5500 lines total)
Test Files: test_tui.py, test_widget.py
Overall Status: ⚠️ Implemented but under-tested (UI testing is challenging)

REQ-UI-003: Heaven GUI (Desktop App)

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Tauri + React framework	ACTION_PLAN.pdf, README.md	`heaven-gui/`	None	✅	N/A	TypeScript/React tests separate
Workpad management UI	FEATURES.md	`heaven-gui/src/components/`	None	✅	N/A	Functional, needs E2E tests
Monaco code editor	FEATURES.md	`heaven-gui/src/components/CodeEditor.tsx`	None	✅	N/A	Implemented
D3.js commit graph	FEATURES.md	`heaven-gui/src/components/CommitGraph.tsx`	None	✅	N/A	Implemented
Recharts metrics	FEATURES.md	`heaven-gui/src/components/TestDashboard.tsx`	None	✅	N/A	Implemented
AI chat panel	FEATURES.md	`heaven-gui/src/components/AIAssistant.tsx`	None	⚠️	N/A	Partial implementation
Command palette (Cmd+K)	FEATURES.md	`heaven-gui/src/components/Command-`	None	✅	N/A	Implemented

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/ Notes
		`Palette.tsx`				
Settings panel	FEATURES.md	`heaven-gui/src/components/SettingsPanel.tsx`	None	⚠️	N/A	Partial implementation
5 engagement levels	ARCHITECTURE.md	`heaven-gui/src/App.tsx`	None	⚠️	N/A	Partial implementation
Write operations	ACTION_PLAN.pdf	`heaven-gui/src-tauri/src/commands.rs`	None	⚠️	N/A	Partial (read-only mostly)

Directory: `heaven-gui/` (Tauri + React)
Test Files: None (needs E2E tests with Playwright)
Overall Status: ⚠️ Functional but needs E2E tests & write operations completion

---

Configuration System

REQ-CONFIG-001: Configuration Management

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
YAML-based configuration	FEATURES.md	`config/manager.py::ConfigManager`	`tests/test_config.py`	✅	High (85%)	None
Environment variable support	FEATURES.md	`config/manager.py::_load_from_env()`	`tests/test_config.py`	✅	High (85%)	None
Multi-profile support	FEATURES.md	`config/manager.py::load_profile()`	`tests/test_config.py`	⚠️	Medium (60%)	Profile switching needs tests
Template system	FEATURES.md	`config/templates.py`	`tests/test_config.py`	✅	High (80%)	None
Secure credential storage	FEATURES.md	`config/manager.py::_encrypt_credentials()`	`tests/test_config.py`	🔴	None	Not implemented yet

Module: `sologit/config/manager.py` (791 lines)
Classes: ConfigManager, SoloGitConfig, ModelConfig, BudgetConfig
Test Files: test_config.py
Overall Status: ✅ Mostly Complete, needs encryption & profile tests

---

REQ-CONFIG-002: Configuration Commands

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
`config init`	README.md	`cli/config_commands.py::init()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed
`config show`	README.md	`cli/config_commands.py::show()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed
`config set`	README.md	`cli/config_commands.py::set_config()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed
`config get`	README.md	`cli/config_commands.py::get_config()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed
`config test`	README.md	`cli/config_commands.py::test_config()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed
`config templates`	README.md	`cli/config_commands.py::list_templates()`	`tests/test_config_commands.py`	⚠️	Low (30%)	CLI tests needed

Module: `sologit/cli/config_commands.py` (363 lines)
Test Files: test_config_commands.py (needs expansion)
Overall Status: ⚠️ Implemented but under-tested

State Management

REQ-STATE-001: State Schema & Storage

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
JSON-based state file	ARCHITECTURE.md	`state/manager.py::JSONStateBackend`	`tests/test_state_manager.py`	✅	High (85%)	None
Repository state tracking	ARCHITECTURE.md	`state/schema.py::RepositoryState`	`tests/test_state_schema.py`	✅	High (90%)	None
Workpad state tracking	ARCHITECTURE.md	`state/schema.py::WorkpadState`	`tests/test_state_schema.py`	✅	High (90%)	None
Test result history	ARCHITECTURE.md	`state/schema.py::TestRun`	`tests/test_state_schema.py`	✅	High (90%)	None
AI operation history	ARCHITECTURE.md	`state/schema.py::AIOperation`	`tests/test_state_schema.py`	✅	High (90%)	None
Event system	ARCHITECTURE.md	`state/schema.py::StateEvent`	`tests/test_state_manager.py`	⚠️	Medium (65%)	Event tests needed

Modules: `sologit/state/manager.py` (768 lines), `schema.py` (260 lines)
Classes: StateManager, JSONStateBackend, multiple schema classes
Test Files: test_state_manager.py, test_state_schema.py
Overall Status: ✅ Core complete, needs event tests

REQ-STATE-002: Git ↔ State Sync

Requirement	Spec Source	Implementation	Tests	Status	Coverage	Gaps/Notes
Bidirectional sync	ARCHITECTURE.md	`state/git_sync.py::GitStateSync`	`tests/test_git_sync.py`	✅	Medium (75%)	None
Commit detection	ARCHITECTURE.md	`state/git_sync.py::detect_changes()`	`tests/test_git_sync.py`	✅	Medium (75%)	None
State recovery on corruption	FEATURES.md	`state/git_sync.py::recover_state()`	`tests/test_git_sync.py`	⚠️	Low (50%)	Recovery tests needed
State backup	FEATURES.md	`state/git_sync.py::backup_state()`	`tests/test_git_sync.py`	⚠️	Low (50%)	Backup tests needed
Conflict resolution	ARCHITECTURE.md	`state/git_sync.py::resolve_conflicts()`	`tests/test_git_sync.py`	⚠️	Low (45%)	Conflict tests needed

Module: `sologit/state/git_sync.py` (591 lines)
Test Files: test_git_sync.py
Overall Status: ⚠️ Core complete, needs recovery & conflict tests

---

Coverage Summary

By Module Category

Category	Modules	Total Lines	Avg Coverage	Test Files	Status
Core	2	239	95%	2	✅ Excellent
Engines	3	3,275	92%	15+	✅ Excellent
Orchestration	13	3,236	84%	20+	✅ Good
Analysis	1	426	90%	2	✅ Excellent
Workflows	4	1,338	72%	8	⚠️ Needs Improvement
State	3	1,619	77%	6	✅ Good
Config	2	931	83%	3	✅ Good
CLI	6	5,419	35%	5	🔴 Critical - Needs Tests
UI	14	5,636	48%	10+	⚠️ Needs Improvement
Utils	1	130	80%	2	✅ Good
API	1	589	31%	3	🔴 Critical - Needs Tests

Overall Project Stats

- **Total Source Files**: 46 Python modules
- **Total Lines of Code**: ~22,800 lines
- **Total Test Files**: 61 test files
- **Total Tests**: ~555 tests
- **Overall Coverage**: 76% (per README)
- **Core Components Coverage**: 90%+

High-Priority Gaps

Critical (Immediate Action Required)

1. **CLI Integration Tests** - Only 35% coverage
   - Need functional tests for all commands
   - Need JSON/CSV output validation
   - Need error handling tests

2. **API Client Tests** - Only 31% coverage
   - Need mock API call tests
   - Need error handling tests (network, auth, rate limits)
   - Need streaming tests

3. **UI Component Tests** - 48% average coverage
   - TUI widgets need more tests
   - Keyboard navigation tests
   - Event handling tests

Important (Next Phase)

1. **Workflow Edge Cases** - 72% coverage
   - More rollback scenarios
   - More CI integration scenarios
   - More conflict resolution tests

2. **State Sync Edge Cases** - 77% coverage
   - Corruption recovery tests
   - Conflict resolution tests
   - Backup/restore tests

3. **Configuration Edge Cases** - 83% coverage
   - Profile switching tests
   - Encryption tests (not implemented)
   - Template validation tests

Feature Completeness

✅ Fully Implemented & Tested (>85% coverage)

- Core repository management
- Workpad lifecycle
- AI model routing
- Planning & code generation
- Cost management
- Test orchestration
- Test analysis

⚠️ Implemented but Under-Tested (50-85% coverage)

- Auto-merge workflow
- Promotion gate
- CI orchestration
- Rollback handler
- State synchronization
- Configuration system
- UI components (TUI)

🔴 Missing or Critical Gaps (<50% coverage)

- CLI command integration tests
- API client comprehensive tests
- GitHub Actions integration (not implemented)

- Security scanning hooks (not implemented)
- Notification system (not implemented)
- Credential encryption (not implemented)

Recommendations for Phase 2 Refactor

Priority 1: Test Coverage Improvements

1. **CLI Integration Tests**
   - Add Click CliRunner tests for all commands
   - Test JSON/CSV output formats
   - Test error scenarios and help messages

2. **API Client Tests**
   - Mock all external API calls
   - Test error handling comprehensively
   - Test retry logic and timeouts

3. **UI Component Tests**
   - Add more Textual widget tests
   - Test keyboard navigation
   - Test event propagation

Priority 2: Code Consolidation

1. **Duplicate Code Detection**
   - Multiple CLI command files could be consolidated
   - Test utility functions could be centralized
   - Error handling patterns could be unified

2. **Naming Conventions**
   - Standardize function naming (e.g., get vs fetch vs retrieve)
   - Consistent error class naming
   - Uniform logging patterns

3. **Interface Boundaries**
   - Clearer separation between CLI and core logic
   - Better abstraction for external services
   - Consistent return types across modules

Priority 3: Missing Features (Deferred)

1. GitHub Actions integration
2. Security scanning hooks
3. Notification system
4. Credential encryption
5. Command aliases
6. GUI write operations completion

Change Log

- **2025-11-03**: Initial coverage matrix created
- **2025-11-03**: Fixed pytest.ini duplicate configuration

Next Steps

1. **Gap Analysis** - Detailed analysis of each gap
2. **Refactor Plan** - Safe refactoring without behavior changes
3. **Pre-Flight Test Suite** - Comprehensive test battery
4. **CI Pipeline Setup** - Automated testing infrastructure
5. **Make Commands** - Convenience commands for audit, refactor, test
6. **Audit Report** - Final comprehensive report

This coverage matrix will be continuously updated as the Phase 2 audit progresses.